



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

PROGRAMOZÁSI NYELVEK ÉS FORDÍTÓPROGRAMOK

TANSZÉK

Transzpilált nyelv koncepcióigazolása fordítási idejű invariánsokkal

Témavezető:

Bense Viktor

doktorandusz

Szerző:

Suszter Roland

programtervező informatikus BSc

Budapest, 2026

SZAKDOLGOZAT TÉMABEJELENTŐ

Hallgató adatai:

Név: Suszter Roland
Neptun kód: XJQ2DB

Képzési adatok:

Szak: programtervező informatikus, alapképzés (BA/BSc/BProf)
Tagozat : Nappali
Belső témavezetővel rendelkezem

Témavezető neve: Bense Viktor

munkahelyének neve, tanszéke: ELTE IK, Programozási nyelvek és Fordítóprogramok Tanszék
munkahelyének címe: 1117, Budapest, Pázmány Péter sétány 1/C.
beosztás és iskolai végzettsége: doktorandusz, Msc

A szakdolgozat címe: Transzpilált nyelv koncepcióigazolása fordítási idejű invariánsokkal

A szakdolgozat témája:

(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben szakdolgozat témájának leírását)

Manapság az informatikának egyre több szektorában jelenik meg az az igény, hogy a kód ne csak hatékony, hanem helyes is legyen. Ugyanakkor igen gyakran egyensúlyt kell keresnünk a program hatékonysága és a helyesség ellenőrzése között, mert akárhányszor egy függvénynek elő- vagy utófeltételt szeretnénk megfogalmazni és azokat kódban ellenőrizni, illetve a felhasználóval betartatni azokat, valamennyi teljesítményt fel kell áldozni ezek ellenőrzéséhez, melyet nem minden esetben tehetünk meg. Több olyan szituáció is előfordul, ahol egy erősebb heurisztika megfogalmazásával hatékonyabb kódot tudnánk írni, de az előfeltétel ellenőrzésének szükségessége miatt a kód futtatása mégis lassabb lenne. Ennek a problémának megoldásaként merült fel egy olyan programozási nyelv létrehozása, mely ezeket az elő-, utófeltételeket, invariánsokat, ha lehetséges, akkor fordítási időben, ha nem, akkor futtatáskor egyszer a lehető leghamarabb ellenőrzi azokat. Ezt a célt egy egyéni függőtípusos típusrendszert használva tudjuk elérni. A nyelvnek nem célja egy átlag programozótól elvárni a függvényének helyességének bizonyítását különböző szintaktikai és szemantikai eszközökkel, mert ez kimondottan nagy mértékben lassítaná a fejlesztési folyamatot, teljesen új és másfajta kompetenciát is igényel, ami eléggé megnehezítené a nyelvnek a használatát. Jelen projekt célja egy egyszerűbb imperatív nyelv implementálása, mellyel demonstrálható és konzisztensen mérhető a futási időben nyújtott előnyök, esetleges hátrányok más elterjedt programozási nyelvekhez képest.

Budapest, 2025. 10. 02.

Tartalomjegyzék

1. Bevezetés	4
2. Felhasználói dokumentáció	6
2.1. Bevezetés a WIC programozási nyelvbe	6
2.2. Rendszerkövetelmények és környezet kialakítása	7
2.2.1. Hardveres elvárások	7
2.2.2. Szoftveres függőségek és telepítés	7
2.3. A Fordítóprogram (Transzpiler) használata	8
2.3.1. Alapszintaxis és argumentumok	9
2.3.2. A Statikus Analízis Kimenetei	9
2.4. Nyelvi Referencia: Adattípusok és Változók	11
2.4.1. Primitív típusok	11
2.4.2. Módosítószók (Qualifiers)	11
2.4.3. Összetett adattípusok: Tömbök és Mutatók	12
2.4.4. Struktúrák, Uniók és Enumok	13
2.5. Vezérlési szerkezetek és operátorok	13
2.5.1. Feltételes elágazások: If, Else és a Szimbolikus Ágak	13
2.5.2. Ciklusok és a "Kigöngyölítés" (Unrolling)	14
2.6. Szerződésalapú Programozás	14
2.6.1. Függvények Előfeltételei (Preconditions)	14
2.6.2. Függvények Utófeltételei (Postconditions) és a 'res' kulcsszó	15
2.6.3. Ciklus Invariánsok (Loop Invariants) magyarázata	16
2.7. Beépített Biztonsági Ellenőrzések	17
2.7.1. Tömbtúlindekelés védelme (Array Out-of-Bounds)	17
2.7.2. Nullával való osztás (Division by Zero)	17
2.8. Transzpilálás: A Generált C kód és Interoperabilitás	18
2.8.1. Preprocesszor Direktívák Támogatása	18

2.8.2.	A Kimeneti fájl szerkezete	19
2.9.	Gyakorlati jótanácsok a sikeres analízishez	19
3.	Fejlesztői dokumentáció	20
3.1.	Elméleti Háttér	20
3.1.1.	Formális Verifikáció és SMT Megoldók	20
3.1.2.	Szerződésalapú Programozás	20
3.2.	Rendszerarchitektúra és Fejlesztési Környezet	21
3.2.1.	A Fordító Adatáramlása (Pipeline)	21
3.2.2.	Fejlesztési Környezet és Technológiák	22
3.2.3.	Analízis Állapotkezelése (AnalyzerState)	22
3.3.	Lexikális és Szintaktikai Elemzés (Parser)	23
3.3.1.	Az elemzési stratégia megválasztása	23
3.3.2.	Lexikális elemzés és alapszintű kombinátorok	23
3.3.3.	Az Absztrakt Szintaxisfa (AST) felépítése	24
3.3.4.	Kifejezéselemzés és precedencia	25
3.3.5.	Szerződések elemzése	26
3.3.6.	Összetett típusok és globális definíciók	26
3.3.7.	Hibaüzenetek és SourcePos	27
3.4.	Szemantikus Analízis és Memóriamodell	27
3.4.1.	Az Állapottér (State) és a Monád Verem	27
3.4.2.	A <code>StateT</code> Monád	27
3.4.3.	A Szimbolikus Memória Reprezentációja	28
3.4.4.	Virtuális Memória Layout és Címzés	29
3.4.5.	Típusméretek meghatározása (<code>sizeof</code>)	29
3.4.6.	Mező eltolódások (<code>offsetof</code>)	29
3.4.7.	L-érték (<code>lvalue</code>) Feloldás és Memórahivatkozások	30
3.4.8.	Implicit Biztonsági Szerződések	30
3.4.9.	Osztás és tömb túlindexelés elleni védelem	31
3.4.10.	Konkrét Kiértékelés (<code>evalPure</code>)	31
3.5.	Szimbolikus Végrehajtás és Tételbizonyítás	32
3.5.1.	A Szimbolikus Motor (<code>symExec</code>)	32
3.5.2.	Az Üzemanyag (<code>fuel</code>) Paraméter	32
3.5.3.	Elágazások Szimbolikus Kezelése (<code>If-Else</code>)	33

3.5.4.	Útvonalfeltételek (Path Conditions)	33
3.5.5.	Memóriaállapotok Fúziója (Store Merging)	33
3.5.6.	Ciklusok (while) Elemzési Stratégiái	34
3.5.7.	SMT Solver Integráció (proveZ3)	35
3.5.8.	A translateZ3 függvény	35
3.5.9.	A Döntési Algoritmus (proveZ3)	36
3.6.	Kódgenerálás és a WIC Transzpiler	36
3.6.1.	A Kimeneti Fájl Felépítése és C-interoperabilitás	37
3.6.2.	Típusok és Globális Deklarációk Leképezése	37
3.6.3.	Összetett Adatszerkezetek (Struct, Union, Enum)	38
3.6.4.	Minősítők (Qualifiers) és Deklarációk	38
3.6.5.	Kifejezések és Utasítások Fordítása	38
3.6.6.	Kifejezések (Expressions)	38
3.6.7.	Dinamikus Biztonsági Háló: Asszertációk Injektálása	39
3.6.8.	Szerződések futásidejű ellenőrzése	39
3.7.	Szoftvertesztelés és Validáció	40
3.7.1.	A Tesztelési Keretrendszer Architektúrája	40
3.7.2.	Az AST Pozícióttörlési Mechanizmusa	40
3.7.3.	Szimbolikus Analízis és SMT Logika Validációja	41
3.7.4.	Negatív tesztek (Provably False)	41
3.7.5.	Nemlineáris Aritmetika és Eldönthetetlenség	42
4.	Összegzés	43
4.1.	Eredmények összefoglalása	43
4.2.	Jövőbeli fejlesztési lehetőségek	44
	Köszönetnyilvánítás	46
	Irodalomjegyzék	46
	Forráskódjegyzék	48

1. fejezet

Bevezetés

Manapság az informatikának egyre több szektorában jelenik meg az a kritikus elvárás, hogy az elkészült szoftverek ne csupán hatékonyak, hanem bizonyíthatóan helyesek is legyenek. A szoftverfejlesztés során a mérnökök igen gyakran kényszerülnek kompromisszumot kötni a program futási teljesítménye és a helyesség ellenőrzése között. Akárhányszor egy függvénynek szigorú elő- vagy utófeltételt szeretnénk megfogalmazni, majd azokat a futó kódban ellenőrizni (és a felhasználóval betartatni), valamennyi processzoridőt és teljesítményt fel kell áldoznunk ezekre az extra műveletekre. Ezt a teljesítményvesztéséget azonban nem minden környezet engedheti meg magának. Több olyan szituáció is előfordul az iparban, ahol egy erősebb heurisztika vagy egy optimalizált algoritmus megfogalmazásával jóval hatékonyabb kódot tudnánk írni, de a kötelező futásidejű biztonsági ellenőrzések miatt a program végső soron mégis lassabbá válna.

Erre a problémára nyújthat kézenfekvő megoldást egy olyan programozási nyelv létrehozása, amely az említett elő- és utófeltételeket, valamint a ciklusinvariánsokat, amennyiben az matematikailag lehetséges, már fordítási időben leellenőrzi. Ezzel a módszerrel a helyes kód garantáltan mentesül a futásidejű overhead alól. Ha pedig az adott invariáns a fordítás során nem bizonyítható (mert például egy futásidőben beolvasott értéktől függ), a rendszer képes arra, hogy a futtatás során a lehető leghamarabb végezze el ezt az ellenőrzést. Bár a modern funkcionális nyelvekben (mint a Haskell) az erős és statikus típusrendszerek garantálnak egyfajta programbiztonságot, a mi megközelítésünk ezt egy lépéssel továbbviszi: egy egyéni, függőtípusos (*dependent type*) rendszerhez hasonló logikát kívánunk alkalmazni egy egyszerűbb, imperatív környezetben.

A jelen projekt célja egy transzpilált imperatív programozási nyelv, a WIC, koncepcióigazolásának implementálása. Ennek a nyelvnek kifejezetten nem célja, hogy az átlagos programozótól elvárja a megírt függvények teljes matematikai helyességének bizonyítását különböző bonyolult szintaktikai és szemantikai eszközökkel. Egy ilyen elvárás ugyanis nagy mértékben lassítaná a szoftverfejlesztési folyamatot, ráadásul teljesen új, a hagyományos programozástól eltérő kompetenciákat igényelne a felhasználótól. Ehelyett a WIC egy olyan imperatív nyelv, amely automatizálja ezt a folyamatot: a forráskódot C nyelvre fordítja át, miközben a statikus analízátora szimbolikus végrehajtást használ a kód bejárására.

A hagyományos parser generátorok (mint például a Happy) és a Backus–Naur forma (BNF) gyakran okoznak nehezen értelmezhető hibaüzeneteket, ezért a WIC szintaktikai elemzője kézzel írt *Recursive Descent* algoritmussal és a Megaparsec könyvtár használatával készült. Az elkészült absztrakt szintaxisfát a statikus elemző dolgozza fel, amely a program memóriaállapotát és a szerződéseket (szerkezeteket, pointereket, ciklusokat) SMT (*Satisfiability Modulo Theories*) lekérdezésekké alakítja, majd a Z3[1] SMT solver segítségével próbálja megcáfolni vagy bizonyítani azokat.

Amennyiben a Z3 képes bizonyítani, hogy egy adott hiba (például nullával való osztás vagy tömb túlindexelés) sosem következhet be, a transzpilált C kódba semmilyen futásidejű ellenőrzés nem kerül be. Ha a Z3 szerint a feltétel bizonyíthatatlan, a fordító figyelmezteti a fejlesztőt, és egy C-szintű `assert` utasítást generál a kódba. Ezzel a módszerrel sikeresen demonstrálhatók és konzisztensen mérhetők a fordítási idejű invariánsok futási időben nyújtott előnyei a más, elterjedt programozási nyelvekkel szemben.

2. fejezet

Felhasználói dokumentáció

2.1. Bevezetés a WIC programozási nyelvbe

A WIC egy statikusan típusos, imperatív programozási nyelv, amelynek szintaktikája tudatosan épít a C nyelv hagyományaira. A nyelv megalkotásának elsődleges motivációja az volt, hogy a szoftverek helyességét és memóriabiztonságát ne kizárólag futásidejű ellenőrzésekkel (mint például `assert`, `exceptions`), illetve utólagos teszteléssel garantáljuk, hanem már a fordítás pillanatában, matematikai pontossággal bizonyítsuk.

A WIC környezet a megszokott fordítókkal (mint a GCC vagy a Clang) ellentétben nem közvetlenül gépi kódot állít elő. Ehelyett a bemeneti `.wic` forrásfájl egy komplex statikus analízisnek és szimbolikus végrehajtásnak veti alá. Ezen folyamat során a háttérben dolgozó Microsoft Z3 Theorem Prover megvizsgálja a program lehetséges futási útvonalaait, és megkísérli bizonyítani a programozó által definiált invariánsokat és biztonsági szabályokat. Sikeres bizonyítás esetén a WIC fordító standard, optimalizált C kódot generál, amely aztán bármely platformon natívan lefordítható, más projekt részeként felhasználható.

Ez a megközelítés lehetővé teszi, hogy a szoftverfejlesztő nulla futásidejű költséggel érjen el magas szintű memóriabiztonságot azokon a pontokon, amelyeket az SMT solver sikeresen tudott verifikálni. Ugyanakkor hibrid megoldásként, ha egy megszorítás matematikailag túlságosan bonyolult, a rendszer biztonsági hálóként futásidejű ellenőrzéseket generál.

Mindezen felül a WIC képes saját utófeltételeket és invariánsokat megfogalmazni az egyes függvényeknek, melyek bizonyíthatóan teljesülnek, ami könnyedséget

okoz a felhasználónak, hogy ne kelljen minden triviális feltételeket megfogalmazniuk. Ugyanakkor ez a képesség segíti a futtatási időben történő ellenőrzések elkerülését.

2.2. Rendszerkövetelmények és környezet kialakítása

A WIC fordító megfelelő működéséhez több különböző technológiai réteg együttműködése szükséges. A fejlesztői környezet felépítése során figyelemmel kell lenni a nyelv alapjául szolgáló Haskell futtatókörnyezetre és az SMT solver natív könyvtárára.

2.2.1. Hardveres elvárások

Bár a WIC programok írásához bármilyen egyszerű szövegszerkesztő elegendő, a fordítás (statikus analízis) számításigényes folyamat. A szimbolikus végrehajtás memóriaigénye a kód bázis összetettségével, az elágazások számával és a ciklusok bonyolultságával exponenciálisan nőhet.

- **Processzor (CPU):** Legalább 2 magos, modern x86_64 vagy ARM processzor javasolt a Z3 solver gyors lefutásához.
- **Memória (RAM):** Minimum 4 GB, de nagyobb, komplex elő- és utófeltételekkel rendelkező projektek esetén 8 GB vagy annál több rendszermemória ajánlott a solver túlzott lapozásának (swapping) elkerülése végett.

2.2.2. Szoftveres függőségek és telepítés

A telepítési folyamat platformfüggő. A három legfőbb külső komponens a Haskell, Cabal és a Z3 Theorem Prover.

Telepítés Linux (Debian/Ubuntu) környezetben

Linux alatt a telepítés rendkívül gyorsan elvégezhető a beépített csomagkezelők segítségével. Fontos, hogy a Z3 esetén a fejlesztői fejlécfájlok (headers) is telepítésre kerüljenek, mivel a Haskell-Z3 (z3 csomag) ezeken keresztül kommunikál.

```
1 # 1. Z3 solver és fejlesztői könyvtárak
2 sudo apt-get update
3 sudo apt-get install z3 libz3-dev
```

```
4
5 # 2. GHC és Cabal telepítése GHCup segítségével
6 curl --proto '=https' --tlsv1.2 -sSf https://get-ghcup.haskell.org
  | sh
```

2.1. forráskód. Csomagok telepítése Ubuntu alatt

A sikeres csomagtelepítés után a WIC projekt gyökerében a következő paranccsal indítható el a fordítás:

```
1 cabal update
2 cabal install
```

Telepítés Windows környezetben

Windows operációs rendszeren a külső C/C++ könyvtárak (mint a libz3) integrációja a Haskell környezetbe manuális konfigurációt igényel.

1. Töltse le a Z3 legújabb Windows bináris (pre-compiled) kiadását a hivatalos GitHub tárolóból.
2. Csomagolja ki egy elérhető helyre (például `C:\z3`).
3. Másolja a `C:\z3\bin` mappában található `libz3.dll` fájlt egy olyan mappába, amely szerepel a rendszer `PATH` környezeti változójában.
4. A WIC projekt gyökérkönyvtárában hozzon létre egy `cabal.project.local` nevű fájlt, amely megmondja a Cabalnak, hol találja a függőségeket:

```
1 package z3
2   extra-include-dirs: C:\z3\include
3   extra-lib-dirs: C:\z3\bin
```

2.2. forráskód. `cabal.project.local` beállítása Windows alatt

Ezek után a `cabal install` parancs hiba nélkül képes előállítani a futtatható binárist.

2.3. A Fordítóprogram (Transzpiler) használata

A fordítóprogram egyszerű parancssori interfésszel (CLI) rendelkezik. Elsődleges funkciója a WIC forráskód beolvasása, elemzése, majd a verifikált C kód legene-

rálása. A fordításra kettő módszer áll rendelkezésre. Az első a futtatható bináris közvetlen meghívása, a második pedig a `cabal` segítségével lehetséges.

2.3.1. Alapszintaxis és argumentumok

A program indításához legalább két paramétert kell megadni: a bemeneti fájlt és a kívánt kimeneti fájl nevét.

```
1 wic <bemeneti_forras.wic> <kimenet.c>
2 cabal run -- <bemeneti_forras.wic> <kimenet.c>
```

Például egy szenzor adatokat feldolgozó fájl fordítására a következő parancsot használhatjuk:

```
1 wic sensor_logic.wic sensor_logic.c
2 cabal run -- sensor_logic.wic sensor_logic.c
```

A transzpilier futása során az alábbi fázisokon megy keresztül, amelyek mindegyike különböző típusú hibákat tárhat fel:

1. **Preprocesszálás:** A forráskódból kivonásra kerülnek a C-specifikus preprocessor direktívák (pl. `#include <stdio.h>`), hogy azok ne zavarják a WIC saját szintaktikai elemzőjét.
2. **Lexikális és szintaktikai elemzés (Parsing):** A Haskell Megaparsec könyvtárára épülő elemző felépíti az Absztrakt Szintaxisfát (AST). Itt derülnek ki a hiányzó pontosvesszők, elért kulcsszavak és zárójel-párosítási hibák. Szintaktikai hiba esetén a fordítás azonnal megszakad.
3. **Statikus és Szimbolikus Analízis:** Ez a szakasz a nyelv legfontosabb része. A fordító bejárja az AST-t, és minden függvényt, változót, elágazást egy matematikai formulává alakít, amelyet átad a Z3 solvernek.
4. **Kódgenerálás:** Amennyiben a logikai feltételek és az analízis megfelelő eredménnyel zárul, elkészül a standard C forráskód.

2.3.2. A Statikus Analízis Kimenetei

A WIC programozóknak kiemelt figyelmet kell fordítaniuk a terminálban megjelenő üzenetekre, mivel ezek jelzik a kód biztonsági szintjét. A Z3 SMT solver visszatérési értékeitől függően az analízis háromféle végkifejletet azonosíthat.

1. Bizonyítottan helyes kód (Proven True)

Ebben az ideális esetben a fordító néma marad, vagy sikeres fordítást jelez, és létrejön a kimeneti `.c` fájl. Ez azt jelenti, hogy a solver minden egyes lehetséges bemeneti kombinációra bizonyította, hogy:

- Nem történik tömb-túindexelés.
- Nem történik nullával való osztás.
- A függvények elő- és utófeltételei sosem sérülnek.

A generált C kód itt mentes lesz minden futásidejű ellenőrzéstől.

2. Bizonyíthatatlan megkötések (Runtime Check / WARNING)

Előfordulnak helyzetek, amikor az SMT solver matematikai korlátokba ütközik (például nem-lineáris aritmetika, bitműveletek, vagy bonyolult kriptográfiai algoritmusok során). Ilyenkor a Z3 nem tudja eldönteni, hogy egy kifejezés igaz vagy hamis.

Ilyen esetben a WIC **nem szakítja meg a fordítást**, hiszen a kód még lehet helyes, csupán a fordító nem elég okos hozzá, hogy ezt bizonyítsa. Ehelyett egy figyelmeztetés jelenik meg:

```
1 WARNING: 2 unprovable constraints found.
```

A fordító az ilyen eldönthetetlen állapotokat regisztrálja, és a leggenerált C kód megfelelő pontjaira futásidejű `assert()` hívásokat injektál. Ezzel a hibrid megközelítéssel biztosítható, hogy a szoftver sose fusson tovább korrupt memóriaállapottal, vagy végezzen el olyan műveletet, amely bizonyíthatóan hibás lenne.

3. Bizonyíthatóan hibás kód (Provably False / ERROR)

Ha az analízis talál egyetlen olyan futási utat, ahol egy elvárás (például egy utófeltétel) **biztosan** megsérül, a transzpiller hibaüzenettel azonnal leáll, és nem generál kimenetet.

```
1 ERROR: found provably false states/contracts:
2   -> core.wic:42:5 - Precondition Failed in: setSpeed: val >= 0
```

Ez az üzenet arról tájékoztat, hogy a `core.wic` 42. sorában meghívott `setSpeed` függvény paramétere bizonyíthatóan negatív lehet valamelyik hívási ágon, ami megsérti annak előfeltételét. Ezt a programozónak a forráskódban kell orvosolnia.

2.4. Nyelvi Referencia: Adattípusok és Változók

A WIC típusrendszere szigorúbb és kevesebb implicit konverziót enged meg, mint a hagyományos C nyelv, ezzel is redukálva a rejtett programozási hibák számát.

2.4.1. Primitív típusok

Minden változót explicit típusmegadással kell deklarálni.

- **Int:** Előjeles, platformfüggetlen egész szám.
- **Double:** Lebegőpontos számok. Bár az SMT solverok képesek kezelni a lebegőpontos aritmetikát (IEEE 754 szabvány alapján), ezek bizonyítása gyakran nagyon lassú vagy Unknown eredménnyel zárul, így használatuk a kritikus szerződésekben csak óvatosan javasolt.
- **Bool:** A C-vel ellentétben a logikai értékek nem kezelhetők **Int**-ként (a 0 és a **false** nem felcserélhető). Értékei kizárólag **true** vagy **false** lehetnek.
- **Char:** Egyetlen karakter tárolására alkalmas típus.
- **Void:** Kizárólag függvények visszatérési típusaként alkalmazható, ha a függvény nem ad vissza értéket.

2.4.2. Módosítósók (Qualifiers)

A WIC két fő módosítót támogat változók deklarálásakor:

- **static:** Hatókört és élettartamot módosít. A C nyelvhez hasonlóan a statikus változók a program teljes futása alatt megőrzik értéküket.
- **const:** Kiemelten fontos kulcsszó a WIC esetében. Egy **const** változó értéke garantáltan változatlan marad. A statikus analízis során a konstansok használata jelentősen felgyorsítja a bizonyítási időt, mivel az SMT solvernek kevesebb állapottérrel kell számolnia.

```
1 Int age = 25;
2 const Double PI = 3.14159;
3 static Bool isInitialized = false;
```

2.3. forráskód. Változók és konstansok deklarálása

2.4.3. Összetett adattípusok: Tömbök és Mutatók

A memóriabiztonság szempontjából a tömbök és mutatók (pointerek) a leginkább sérülékeny pontok a C-szerű nyelvekben. A WIC ezt a problémát az analízis szigorításával kezeli.

Tömbök: A WIC támogatja a statikus és dinamikus méretű tömböket. Tömbök esetén a fordító a háttérben fenntart egy úgynevezett memóriamodellt. Amikor a programozó a `buffer[i] = 10;` utasítást használja, a WIC belső reprezentációja az SMT Array elméletét alkalmazva rögzíti az adatot.

Mutatók: A mutatók deklarációja a `*` szimbólummal, a dereferencia (érték kiolvasása) szintén a `*` operátorral, a memóriacím lekérdezése pedig az `&` jellel történik, a C-hez hasonlóan.

```
1 Int data[100];
2 Int *ptr = &data[0];
3 *ptr = 42; // A data[0] értéke 42 lesz
```

2.4. forráskód. Tömbök és mutatók használata

Fontos megjegyzés: A komplex mutatóaritmetika (pl. `ptr = ptr + 15;`) csak akkor megy át hiba nélkül a fordítón, ha a szimbolikus analízis biztosan képes bizonyítani, hogy az új memóriacím továbbra is a kiosztott memóriaterületen (például a `data` tömbön) belül marad.

2.4.4. Struktúrák, Uniók és Enumok

A nyelv támogatja a felhasználó által definiált típusokat. Ezeket az összetett típusokat globális (top-level) hatókörben kell deklarálni. A struktúrák mezőire a `.` (pont) és a mutatók esetén a `->` (nyíl) operátorokkal lehet hivatkozni.

```
1 enum TaskState { PENDING, RUNNING, FINISHED };
2
3 struct Task {
4     Int id;
5     enum TaskState state;
6     Int *payload;
7 };
8
9 struct Task myTask;
10 myTask.id = 1;
11 myTask.state = RUNNING;
```

2.5. Vezérlési szerkezetek és operátorok

A vezérlési szerkezetek felelnek a program futásának irányításáért. Bár szintaktikailag teljesen megegyeznek a C nyelvvel, az analízátor működése szempontjából mélyebb megértést igényelnek a felhasználatól.

2.5.1. Feltételes elágazások: If, Else és a Szimbolikus Ágak

A WIC nyelv megköveteli, hogy minden `if` és `else` blokk tartalma kötelezően kapcsos zárójelek `{ ... }` közé kerüljön. Ezzel elkerülhetők az elcsúszott indentációból fakadó kétértelműségek.

```
1 if (temperature > 100) {
2     criticalError = true;
3 } else {
4     criticalError = false;
5 }
```

Mit csinál az elemző a háttérben? Amikor a WIC statikus analízátora egy elágazáshoz ér, a program állapota (state) úgymond "kettéágazik".

- Az igaz ágon a solver hozzáadja az aktuális tudásbázisához a `(temperature > 100)` állítást. Bármilyen művelet, amit ebben a blokkban végzünk, ezen a feltételezésen alapul. Ha ideírunk egy `assert(temperature > 0);-t`, az automatikusan "Proven True" lesz.
- A hamis ágon a solver a feltétel negáltját, azaz a `!(temperature > 100)` kifejezést adja a tényekhez.

Ha egy if-ág logikailag elérhetetlen (például: `Int x = 5; if (x > 10) {...}`), a solver azonnal felismeri.

2.5.2. Ciklusok és a "Kigöngyölítés" (Unrolling)

A WIC a C nyelvből ismert `while` és `for` ciklusokat biztosítja. A ciklusok azonban a szimbolikus analízis egyik legismertebb gyengeségei, hiszen egy végtelen iteráció esetén a fordító is végtelen ciklusba eshetne elemzés közben.

Ennek kiküszöbölésére a WIC egy Bounded Model Checking (BMC) technikát alkalmaz, ha a ciklushoz nem adunk explicit invariánst. Ez azt jelenti, hogy a fordító a ciklus magját fix, korlátozott számú alkalommal fejti ki és úgy ellenőrzi. Ha egy hiba az első pár iterációban jelentkezne, a WIC megtalálja, de a nagyon késői hibák bizonyításához más eszköz kell, ciklus invariánsokkal.

2.6. Szerződésalapú Programozás

A WIC nyelv forradalmi újítása a C-hez képest a szerződések (contracts) elsőrangú nyelvi elemként való bevezetése. A fejlesztő nem külső dokumentációba írja le, hogy mit vár el egy függvénytől, hanem közvetlenül a nyelvtan részévé teszi azt. A fordító dolga garantálni, hogy ezek a szerződések ne sérüljenek.

2.6.1. Függvények Előfeltételei (Preconditions)

Az előfeltétel egy elvárás a hívó féllel (a felhasználóval vagy egy másik függvénnyel) szemben. A függvényünk csak akkor garantálja a helyes működést, ha olyan bemenetet kap, amely kielégíti ezt a feltételt.

Az előfeltétel a függvény paraméterlistája után, zárójelek `()` között szerepel.


```
1 Int calculateWithdrawal(Int balance, Int amount)
2     (amount > 0 && amount <= balance)
3     //( ) Az utófeltétel helye, amennyiben nincsen, nem kötelező
4     megadni.
5 {
6     return balance - amount;
7 }
```

2.5. forráskód. Függvény egy egyszerű előfeltétellel

Magyarázat: A `calculateWithdrawal` függvény kiköti, hogy a levonandó összeg nem lehet negatív, és nem haladhatja meg a rendelkezésre álló egyenleget. Ha a programban bárhol máshol leírjuk a `calculateWithdrawal(100, 500)` hívást, a WIC fordító azonnal `Provably False` hibát dob, megakadályozva a kód lefordítását, mivel a $500 \leq 100$ állítás hamis.

2.6.2. Függvények Utófeltételei (Postconditions) és a 'res' kulcsszó

Az utófeltétel a függvény ígérete a kívülág felé, garantálja, hogy miután lefutott, a külső állapot és a visszatérési érték megfelel ennek a kényszernek.

Az utófeltétel a függvény definíciójában a második zárójelpárban foglal helyet. Ha a függvénynek van visszatérési értéke, a speciális `res` kulcsszóval hivatkozhatunk rá.

```
1 Int absoluteValue(Int x)
2     ( ) //Ha van utófeltétel, akkor az előfeltételnek a zárójelpárjába
3     kötelező kiírni, még ha üres, akkor is.
4     (res >= 0)
5 {
6     if (x < 0) {
7         return -x;
8     }
9     return x;
10 }
```

2.6. forráskód. Függvény utófeltétellel

Ebben az esetben a függvénynek nincs előfeltétele (bármilyen `Int` bemehet), de szigorú az utófeltétele: a visszaadott érték (`res`) sosem lehet negatív. Amikor az elemző

ezt a függvényt olvassa, mindkét elágazást (`if` és implicit `else`) megvizsgálja, és matematikai bizonyítást készít arról, hogy a végeredmény tényleg nem lehet kisebb, mint nulla.

Ha a programozó elírná a kódot, és például `return x;` maradna a mínusz helyett a negatív ágon, a WIC fordító *"Postcondition Failed"* üzenettel állna le.

2.6.3. Ciklus Invariánsok (Loop Invariants) magyarázata

Mint korábban említettük, a ciklusok önálló elemzése nehéz. Ezen segít a ciklus invariáns. Az invariáns egy olyan logikai állítás, amely a `while` ciklus feltétele után kerül megadásra, és az elemző arra használja, hogy matematikai indukciót alkalmazzon a programkód felett.

Egy jól megfogalmazott invariánsnak három feltételt kell teljesítenie:

1. **Inicializáció (Alapulvétel):** Igaznak kell lennie közvetlenül azelőtt, hogy a program először belépne a ciklusba.
2. **Induktív Lépés:** Ha igaz volt a ciklusmag végrehajtása előtt, akkor igaznak kell maradnia a ciklusmag egy iterációjának befejezése után is.
3. **Következtetés (Hasznosság):** A ciklus leállási feltételével kombinálva elég információt kell adnia ahhoz, hogy a ciklus után következő kód blokkok és utófeltételek biztonsága bizonyítható legyen.

```
1 Bool copyData(Int src[10], Int dest[10], Int n)
2     (n >= 0 && n <= 10)
3     ()
4 {
5     Int i = 0;
6     // Ciklus-invariáns: i mindig 0 és n közötti zárt
7     // intervallumban van
8     while (i < n) (i >= 0 && i <= n) {
9         dest[i] = src[i];
10        i++;
11    }
12    return true;
```

2.7. forráskód. A ciklus invariáns gyakorlati alkalmazása

Mi történik a háttérben? A Z3 solver nem fejt ki a ciklust, hanem helyette az alábbi csinálja: kezdetben $i = 0$ és az előfeltétel miatt $n \geq 0$, tehát az invariáns ($0 \geq 0$ és $0 \leq n$) igaz. Amikor belép a ciklusba, feltételezi az invariánst és hozzáveszi, hogy a ciklusfeltétel igaz ($i < n$). Végrehatja a magot ($i++$), ami után i új értéke továbbra is kisebb vagy egyenlő lesz n -nel. Mivel az invariáns sosem sérült meg az i -edik lépésben, garantálni tudjuk, hogy a végtelenségig ismételve sem fog megsérülni.

2.7. Beépített Biztonsági Ellenőrzések

Az explicit elő- és utófeltételek mellett a WIC fordító implicit, úgynevezett láthatatlan szerződéseket generál minden egyes memória-hozzáférés és veszélyes aritmetikai művelet köré. Ez mentesíti a programozót az alól, hogy folyamatosan manuális biztonsági ellenőrzéseket írjon.

2.7.1. Tömbtúlindexelés védelme (Array Out-of-Bounds)

A C programozás egyik leggyakoribb és legsúlyosabb biztonsági rése a puffertúlsordulás (buffer overflow), ami abból fakad, hogy a kód olyan memóriaterületet olvas vagy ír, ami nem hozzá tartozik. Minden `array[index]` művelet esetén a WIC automatikusan egy rejtett `assert(index >= 0 && index < sizeof(array))` előfeltételt generál.

```
1 void writeToArray(Int arr[5]) {  
2     Int index = 10;  
3     arr[index] = 42; // FORDÍTÁSI HIBA: Provably false!  
4 }
```

2.8. forráskód. Automatikus index ellenőrzés

Az elemző azonosítja az `index` értékét és azonnal leállítja a fordítást, mivel bizonyíthatóan átléptük az 5-ös határt.

2.7.2. Nullával való osztás (Division by Zero)

Matematikailag értelmezhetetlen és futásidejű összeomlást okozó esemény a nullával való osztás. A WIC minden `/` és `%` (modulo) operátor használatakor generál egy megkötést, miszerint a nevező értéke (jobb oldali operandus) nem lehet egyenlő nullával.

```
1 Int divide(Int a, Int b) {  
2     return a / b; // Figyelmeztetés (Warning): b lehet, hogy nulla!  
3 }
```

Mivel a `divide` függvénynek itt nincs előfeltétele, a WIC az analízis során rájön, hogy a hívó fél átadhat nullát a `b` paraméternek. Mivel ezt nem tudja fordítási időben cáfolni, a generált C kódba beletesz egy futásidejű `assert(b != 0);` ellenőrzést, ezzel megelőzve a nemdefiniált viselkedést (Undefined Behavior). A helyes megoldás itt a programozó részéről az lett volna, ha hozzáad egy `(b != 0)` előfeltételt a függvényhez.

2.8. Transzpilálás: A Generált C kód és Interoperabilitás

Miután a WIC befejezte a forráskód verifikációját, a cél platformra optimalizált C (C99[2] szabványú) kódot generál. Ezen C kód natív lefordításához már a rendszer standard eszköztára használható (pl. GCC).

2.8.1. Preprocesszor Direktívák Támogatása

A statikus analízis során a külső C függvénytárak elemzése lehetetlen vagy irracionálisan lassú feladat lenne, ezért a WIC okos preprocesszálást alkalmaz. A forrásfájlban elhelyezett minden `#` jellel kezdődő sor (mint a `#include <math.h>` vagy a `#define MAX_LIMIT 100`) érintetlenül átkerül a kimeneti C fájl tetejére.

A WIC parser automatikusan kibontja és mellőzi ezeket a sorokat az SMT solver hívása előtt, ezáltal a felhasználó szabadon használhat bármilyen platform-specifikus C könyvtárat a WIC projektjén belül anélkül, hogy annak működését a Z3-nak bizonyítania kéne.

2.8.2. A Kimeneti fájl szerkezete

A generált `.c` fájl mindig importálja az alapvető C funkciókat a szabványos könyvtárakból. A fájl elején látható lesz a következő fejléc:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4 #include <assert.h>
5
6 // --- User Directives ---
7 // (Ide kerülnek a WIC fájlból kiolvasott #include és #define sorok
8 // -----)
```

Bármely olyan szerződés vagy biztonsági megkötés (pl. matematikai egyenlőség), amely túl komplex volt a Z3 számára és Unknown értéket eredményezett a statikus elemzőből, ebbe a kimeneti fájlba fog kerülni `assert(...)` formájában. Azok az állítások, amiket a Z3 "Proven True" státusszal nyugtázott, egyszerűen eltűnnek (Zero-cost abstraction), és semmilyen szinten nem lassítják a végleges lefordított szoftvert.

2.9. Gyakorlati jótanácsok a sikeres analízishez

- **Legyen specifikus az előfeltételekkel:** Ne bízson semmit a véletlenre. Ha egy változó csak pozitív szám lehet a logika szerint, azt rögzítse szerződésben.
- **Minimalizálja a globális állapotot:** A globális változók használata jelentősen növeli a SMT lekérdezések méretét és esélyét annak, hogy a solver "Unknown" értéket ad vissza.
- **Ne bonyolítsa az aritmetikát:** A polinomok és a nem-lineáris szorzások (pl. $x^3 + y^3 == z^3$) a Z3 és az összes SMT solver egyik leggyengébb pontjai. Kerülje ezek használatát az invariánsokban, ha lehetséges, vagy készüljön fel a generált futásidejű asszertációkra.

3. fejezet

Fejlesztői dokumentáció

3.1. Elméleti Háttér

3.1.1. Formális Verifikáció és SMT Megoldók

A projekt elméleti alapját a formális verifikáció és a szimbolikus végrehajtás adja. A WIC nem konkrét értékekkel, hanem szimbolikus változókkal dolgozik, amelyek reprezentálják az összes lehetséges bemeneti értéket.

A háttérben zajló bizonyítási folyamat a Microsoft Z3 Theorem Proverre épül, amely egy SMT (*Satisfiability Modulo Theories*) megoldó. A Z3 feladata, hogy eldöntse, hogy a programozó által definiált logikai megkötések (szerződések) megsérülhetnek-e bármilyen futási útvonalon.

- **Kielégíthetetlen (Unsat):** Ha a megkötések negáltja kielégíthetetlen, akkor a feltétel minden esetben teljesül. Ez a WIC "Proven True" állapota.
- **Kielégíthetőség (Sat):** Ha létezik ellenpélda, a fordító figyelmeztet. Ha garantáltan hamis, akkor hibaüzenetet küld, megakadályozva a hibás kód lefordítását.

3.1.2. Szerződésalapú Programozás

A WIC a szerződésalapú programozást (Design by Contract) integrálja az imperatív világba. A koncepció lényege, hogy a függvények és a hívó fél között egy szoftveres "szerződés" jön létre:

1. **Előfeltétel (Precondition):** A hívó fél kötelessége biztosítani a bemeneti adatok helyességét.
2. **Utófeltétel (Postcondition):** A függvény ígérete a visszatérési érték és az állapotváltozások tekintetében.
3. **Invariáns (Invariant):** Olyan állapotok, amelyeknek ciklusok futása során végig fenn kell állniuk az indukciós bizonyításhoz.

3.2. Rendszerarchitektúra és Fejlesztési Környezet

3.2.1. A Fordító Adatáramlása (Pipeline)

A WIC fordító egy többfázisú pipeline-on keresztül dolgozza fel a forráskódot, amely biztosítja a szigorú ellenőrzést a kódgenerálás előtt. A folyamat a `Main.hs` modulban van koordinálva.

1. Forráskód Előkészítése és Preprocesszálás

A fordító első lépésként beolvassa a `.wic` kiterjesztésű fájlt. Mivel a WIC támogatja a C-stílusú direktívákat (pl. `#include`), a `Main.hs` különválasztja ezeket a sorokat a tényleges kódtól, hogy a szintaktikai elemzőt ne zavarják az idegen szintaxisú elemek.

2. Szintaktikai Elemzés és AST Felépítése

A tiszta forráskódot a `Parser.hs` dolgozza fel. A Megaparsec könyvtár segítségével épül fel az Absztrakt Szintaxisfa (AST), amely minden csomópontjában tárolja a forráskódbeli pozíciót (`SourcePos`). Ez a pozícióinformáció kritikus az analízátor hibaüzeneteihez, mivel pontosan meg kell tudni jelölni, hol sérült meg a matematikai megkötés.

3. Statikus Analízis és Szimbolikus Végrehajtás

Ez a rendszer legfontosabb része, amelyet az `Analyzer.hs` modul valósít meg. Az elemző szimbolikusan végrehajtja a programot, felépíti a memóriamodellt és összegyűjti az útvonalfeltételeket. Minden veszélyes művelet (osztás, tömbindexelés) és explicit szerződés esetén az SMT lekérdezést küld a Z3 felé.

4. Transzpilálás és Kódgenerálás

Amennyiben az analízis nem talált `Provably False` állapotot, a `Transpiler.hs` elvégzi a C kód generálását. Ha egy feltételt a Z3 nem tudott bizonyítani (`Unknown`), akkor futásidejű `assert` ellenőrzést injektál a biztonság érdekében.

3.2.2. Fejlesztési Környezet és Technológiák

A WIC fordító Haskell nyelven íródott, kihasználva a funkcionális programozás nyújtotta előnyöket az AST transzformációk során.

Haskell GHC és Cabal Konfiguráció

A projekt a `WIC.cabal` fájlban definiált struktúrát követi.

- **Nyelvi szabvány:** Az executable és a tesztek a `GHC2024` szabványt használják.
- **Főbb függőségek:**
 - `megaparsec`: A gyors és részletes hibaüzeneteket adó parser-combinator könyvtár.
 - `z3`: Haskell kötések a Microsoft Z3 solverhez.
 - `mtl`: State és Error monádok kezelése az analízátor állapotához.
 - `containers`: Adatstruktúrák a szimbolikus memóriamodellhez (`Data.Map`, `Data.Set`).

A Z3 Solver Integrációja

A Z3 integrációja kritikus pont. A rendszernek szüksége van a Z3 binárisokra és a fejlesztői könyvtárakra. Linux környezetben ez a `libz3-dev` csomagon keresztül érhető el, míg Windows alatt a `cabal.project.local` fájlban kell megadni a `libz3.dll` és a hozzá tartozó header fájlok elérési útjait a sikeres fordításhoz.

3.2.3. Analízis Állapotkezelése (`AnalyzerState`)

Az `Analyzer.hs` fájlban definiált `AnalyzerState` struktúra felelős a fordítási idő alatti memóriaállapot és logikai tények nyomon követéséért.


```
1 data AnalyzerState = AnalyzerState
2   { env      :: SymEnv      -- Változok címei
3     , heap    :: Heap        -- Memoria tartalma
4     , facts   :: [Expr]      -- Logikai tények (Path Conditions)
5     , violations :: [(SourcePos, String, Expr)] -- Hibák
6     , unknowns :: [(SourcePos, String, Expr)] -- Runtime ellenőrz
        ést igénylő elemek
7   }
```

3.1. forráskód. Az AnalyzerState felépítése

Ez az architektúra lehetővé teszi, hogy az analízátor emlékezzen arra, milyen feltételek igazak egy adott ágon, és ezeket kontextusként adja át a Z3-nak a bizonyítás során.

3.3. Lexikális és Szintaktikai Elemzés (Parser)

3.3.1. Az elemzési stratégia megválasztása

A WIC fordító szintaktikai elemzője egy kézzel írt *Recursive Descent* elemző, amely a *Megaparsec* könyvtárra épül. A modern fordítófejlesztésben gyakran alkalmazott generátorokkal (mint a Happy vagy az ANTLR) szemben ez a megközelítés több szempontból is előnyösebb a projekt céljaihoz mérten. Egyrészt a *Megaparsec* kombinátor alapú logikája lehetővé teszi a nyelvtan rendkívül rugalmas és olvasható leírását közvetlenül Haskellben, másrészt a kézzel írt elemzők sokkal finomabb kontrollt biztosítanak a hibaüzenetek generálása felett. A hagyományos generátorok gyakran kriptikus hibaüzeneteket adnak vissza szintaktikai hiba esetén, míg a WIC elemzője pontos *SourcePos* információkat szolgáltat, megjelölve a hiba pontos helyét (fájl, sor, oszlop).

3.3.2. Lexikális elemzés és alapszintű kombinátorok

A lexikális elemzés (lexing) a WIC-ben nem válik el élesen a szintaktikai elemzéstől; a *Megaparsec* *scannerless* jellegéből adódóan a két fázis összefonódik, de logikailag elkülönített kombinátorok kezelik a whitespace-eket és a tokeneket.

Whitespace és kommentkezelés

Az elemző alapját a `sc` (space consumer) függvény adja, amely a `Megaparsec L.space` segédfüggvényét használja. Ez a komponens felelős a szóközök, tabulátorok és újsor karakterek elnyeléséért. Emellett itt kerülnek definiálásra a kommentek is: a WIC támogatja a C-stílusú egysoros (`//`) és blokk kommenteket (`/* ... */`). A blokk kommentek használata lehetővé teszi kódvázlatok vagy dokumentációk elrejtését az elemző előtt.

Lexémák és szimbólumok

A tokenek elemzését két segédfüggvény támogatja:

- `lexeme`: Ez a függvény egy tetszőleges elemzőt csomagol be, és garantálja, hogy az elemzett elem utáni összes whitespace elnyelésre kerüljön.
- `symbol`: Fix karaktersorozatok (mint a `"{"` vagy a `";"`) elemzésére szolgál, szintén automatikus whitespace-kezeléssel.

Az azonosítók (`identifier`) elemzése során a rendszer szigorúan ellenőrzi, hogy a felismert név nem szerepel-e a fenntartott kulcsszavak (`reserved`) listájában. Ez megakadályozza, hogy a programozó olyan változóneveket használjon, mint az `if`, a `while` vagy az `Int`.

3.3.3. Az Absztrakt Szintaxisfa (AST) felépítése

Az elemző kimenete egy Absztrakt Szintaxisfa, amely a program logikai szerkezetét reprezentálja. A WIC AST-je abban tér el a hagyományos fordítókétól, hogy minden csomópont tartalmazza a forráskódbeli pozícióját (`SourcePos`), ami elengedhetetlen a későbbi statikus analízis hibaüzeneteihez.

Típusrendszer (Type)

A WIC típusrendszere tükrözi a nyelv imperatív, de verifikációra kihegyezett jellegét. A `Type` adattípus támogatja a primitív típusokat (`Int`, `Double`, `Bool`, `Char`, `Void`), a mutatókat (`Ptr`), a konstans minősítőket (`ConstT`), valamint az összetett típusokat mint a `Struct`, `Union` és `Enum`. Különlegesség a tömbök kezelése (`Array`), ahol az elemző opcionálisan egy kifejezést (`Expr`) is társíthat a típushoz, reprezentálva a tömb statikus vagy dinamikus méretét.

Kifejezések (Expr)

A kifejezések kiértékelése a fordító egyik legbonyolultabb része. Az AST-ben az Expr típus egy SourcePos és egy ExprNode párosa. Az ExprNode tartalmazza az összes lehetséges műveletet:

- Literálok: egész, lebegőpontos, logikai, karakter és sztring.
- Változóhivatkozások és függvényhívások.
- Postfix műveletek: tömbindexelés, mezőelérés (. és ->), inkrementálás/dekrementálás.
- Prefix műveletek: dereferencia (*), címképzés (&), negáció, előjeles műveletek.
- Bináris műveletek és a ternáris operátor (? :).

Utasítások (Stmt)

Az utasítások (StmtNode) alkotják a függvények törzsét. A WIC támogatja a változódeklarációkat (Decl), értékadásokat (Assign), elágazásokat (If) és ciklusokat (While, For). A While ciklus definíciója kiemelkedő fontosságú: opcionálisan tartalmazhat egy kifejezéslistát (Maybe [Expr]), amely a ciklusinvariánsokat tárolja a verifikációs motor számára.

3.3.4. Kifejezéselemzés és precedencia

A WIC az operatorTable mechanizmust használja a kifejezések precedenciájának és asszociativitásának kezelésére. Ez biztosítja, hogy a matematikai műveletek a megszokott prioritással hajtsódjanak végre (pl. a szorzás megelőzi az összeadást).

```

1 operatorTable =
2   [ [ prefix "-" (UnOp "-"), prefix "!" (UnOp "!")
3     , prefix "*" Deref, prefix "&" AddrOf
4     , prefix "++" PreInc, prefix "--" PreDec ]
5   , [ binary "*" (BinOp "*"), binary "/" (BinOp "/"), binary "%"
6     (BinOp "%") ]
7   , [ binary "+" (BinOp "+"), binary "-" (BinOp "-") ]
8   , [ binary "<=" (BinOp "<="), binary "<" (BinOp "<")
9     , binary ">=" (BinOp ">="), binary ">" (BinOp ">") ]
10  , [ binary "==" (BinOp "=="), binary "!=" (BinOp "!=") ]

```

```
10 , [ binary "&&" (BinOp "&&") ]  
11 , [ binary "||" (BinOp "||") ] ]
```

3.2. forráskód. Operátor táblázat a Parser.hs-ben

A precedencia táblázatban felülről lefelé csökken a prioritás. Az elemző a legmagasabb rendű prefix operátoroktól halad a logikai diszjunkció (`||`) felé. A ternáris operátort a rendszer különálló szabályként kezeli, mivel az nem illeszthető be tisztán a táblázatba.

3.3.5. Szerződések elemzése

A WIC fordító egyedi vonása, hogy a függvénydefiníciók közvetlen részévé teszi a matematikai megkötéseket. A `pFunction` elemző felelős a függvény fejlécének és törzsének feldolgozásáért.

- **Előfeltételek (Preconditions):** A függvény argumentumlistája utáni opcionális zárójelben megadott kifejezések listája. Az elemző ezeket a `funcInvars` mezőbe menti.
- **Utófeltételek (Postconditions):** Az előfeltételek utáni második opcionális zárójelpárban megadott kifejezések. Itt a `res` kulcsszó hivatkozhat a visszatérési értékre.

Ezek a szerződések nem csupán kommentek; az elemző valid kifejezésfákká (AST) alakítja őket, amelyeket a statikus analízátor később a Z3 tételbizonyítónak ad át.

3.3.6. Összetett típusok és globális definíciók

A WIC forrásfájl legfelsőbb szintjén (`pFile`) globális változódeklarációk, függvénydefiníciók és összetett típusok (`struct`, `union`, `enum`) szerepelhetnek.

A struktúrák elemzésekor a `pStructDef` beolvassa a struktúra nevét és a kapcsos zárójelek közötti mezőket, ahol minden mező egy típusból és egy névből áll. Az enumok esetén (`pEnumDef`) az elemző az azonosítók listáját tárolja el. Ezek a definíciók globálisan elérhetővé válnak az összes függvény számára, lehetővé téve az összetett adatszerkezetek szimbolikus verifikációját is.

3.3.7. Hibaüzenetek és SourcePos

A Megaparsec `getSourcePos` függvényének használata kritikus a fejlesztői élmény szempontjából. Minden alkalommal, amikor az elemző egy új utasítást vagy kifejezést kezd feldolgozni, rögzíti az aktuális pozíciót. Ha az elemzés sikertelen, a rendszer képes megmondani, hogy melyik fájl hányadik sorában történt a hiba. Ha az elemzés sikeres, de a statikus analízátor később ellentmondást talál (például egy előfeltétel megsérül), ezeket a pozíciókat használja a hibaüzenet pontosításához: `core.wic:42:5 - Precondition Failed`.

3.4. Szemantikus Analízis és Memóriamodell

A szintaktikai elemzés során felépített Absztrakt Szintaxisfa (AST) önmagában csupán a forráskód strukturális reprezentációja. Ahhoz, hogy a WIC fordító képes legyen matematikai bizonyításokat végezni a kód helyességére vonatkozóan, az AST-t egy precíz szemantikus modellé kell alakítania. Az **Analyzer.hs** modul felelős a C-szerű memória-kiosztási szabályok leképezéséért, a típusellenőrzésért és a programállapot nyomon követéséért a szimbolikus végrehajtás során.

3.4.1. Az Állapottér (State) és a Monád Verem

A statikus analízis alapvetően egy állapotgépként fogható fel, amely utasításról utasításra haladva frissíti a virtuális memória tartalmát és a bizonyított logikai tények listáját. Ezt a WIC egy monád transzformátor veremmel (*Monad Transformer Stack*) valósítja meg.

3.4.2. A StateT Monád

Mivel az analízátornak nem csupán a belső állapotot kell módosítania, hanem a háttérben futó Microsoft Z3 SMT solverrel is kommunikálnia kell (amely IO műveleteket igényel), a rendszer a `StateT AnalyzerState IO` a típuszínónimát használja, amelyet **Analyzer**-nek nevez. Ez az architektúra biztosítja, hogy a mellékhatások (például a solver hívások) tisztán elkülönüljenek az állapotváltozásoktól.

3.4.3. A Szimbolikus Memória Reprezentációja

Az analízátor állapotát az `AnalyzerState` rekord típus fogja össze, amely részletesen modellezi a fizikai számítógép memóriáját.

```

1 type Addr = Int
2 type SymEnv = M.Map String Addr
3 type Heap = M.Map Addr Value
4 type SymStore = M.Map Addr Expr
5
6 data AnalyzerState = AnalyzerState
7   { env      :: SymEnv
8   , heap     :: Heap
9   , heapTypes :: M.Map Addr Type
10  , structs  :: M.Map String [(Type, String)]
11  , unions   :: M.Map String [(Type, String)]
12  , nextAddr :: Addr
13  , facts    :: [Expr]
14  , funcs    :: M.Map String Function
15  , violations :: [(SourcePos, String, Expr)]
16  , unknowns  :: [(SourcePos, String, Expr)]
17  , returnVal :: Maybe Value
18  , summaries :: M.Map String FuncSummary
19  } deriving (Show)

```

3.3. forráskód. Az AnalyzerState felépítése

A memóriamodell három fő rétegre oszlik:

1. **Környezet (env):** Összerendeli a forráskódban szereplő változóneveket (pl. "x") egy virtuális memóriacímmel (`Addr`).
2. **Kupac (heap):** A tényleges memóriaterület, amely a címeket konkrét értékekhez (`Value`) rendeli. Az értékek lehetnek egészek (`VInt`), memóriacímek (`VAddr`), vagy akár ismeretlen, szimbolikus értékek (`VUnknown`).
3. **Típus-térkép (heapTypes):** Nyilvántartja, hogy egy adott memóriacímen milyen adattípus helyezkedik el, ami a mutatóaritmetikához és a memória-elérés ellenőrzéséhez kritikus.

A `facts` mező tárolja az úgynevezett *Path Condition*-öket (útvonalfeltételeket), azaz azokat a logikai állításokat, amelyek az aktuális végrehajtási ponton garantáltan igazak (pl. egy `if (x > 0)` igaz ágán az `x > 0` tényként szerepel).

3.4.4. Virtuális Memória Layout és Címzés

A C nyelv szemantikájának megfelelően a WIC memóriamodelljének is kezelnie kell az összetett adattípusok (struktúrák, uniók, tömbök) memóriában elfoglalt helyét. Mivel egy struktúra mezőire értékadás történhet (`pt.x = 10`), a fordítónak tudnia kell a pontos bájt-eltolódásokat (offset).

A műveletek feldolgozásának megkönnyítésére az analízátor bevezeti a `Layout` adatstruktúrát, amely az aktuális `struct`, `union`, `heapTypes` és `env` állapotok egy statikus pillanatképe.

3.4.5. Típusméretek meghatározása (`sizeof`)

A `sizeof` függvény rekurzívan kiszámítja egy adattípus virtuális méretét.

- Primitív típusok (`Int`, `Double`, `Bool`, `Ptr`) 1 egységnyi méretet foglalnak el.
- Tömbök (`Array`) mérete az elemek számának és a belső típus méretének szorzata (feltéve, hogy a méret fordítási időben ismert, azaz `IntLit`).
- Struktúrák (`Struct`) esetén a rendszer megkeresi a definíciót az állapotban és összegzi a benne lévő mezők méreteit.
- Uniók (`Union`) esetében a legnagyobb méretű mező méretét veszi fel az unió teljes mérete.

3.4.6. Mező eltolódások (`offsetof`)

Amikor a kód egy struktúra egy specifikus mezőjére hivatkozik, az `offsetof` függvény kiszámítja, hogy a struktúra kezdőcímétől számítva mennyit kell "lépni" a memóriában.

```
1 offsetof :: Layout -> Type -> String -> Maybe Int
2 offsetof l ty prop = case ty of
3     Struct n -> do
4         fields <- M.lookup n (lStructs l)
5         let go [] = Nothing
6             go ((t, name):fs)
7             | name == prop = Just 0
8             | otherwise    = (+ sizeof l t) <$> go fs
9     go fields
```

```
10 Union _ -> Just 0
11 -- ...
```

3.4. forráskód. Az `offsetOf` függvény

Uniók esetén az eltolódás mindig 0, hiszen minden mező ugyanazon a memóriacímen osztozik. Ezzel a megoldással a WIC hűen szimulálja a C nyelv adatszerkezeteinek viselkedését, beleértve az aliasing jelenségeket is.

3.4.7. L-érték (lvalue) Feloldás és Memóriahivatkozások

Egy imperatív nyelvben az értékadó utasítások bal oldalán (LHS) szereplő kifejezéseknek feloldhatónak kell lenniük egy konkrét memóriacímre (l-érték). Az `lvalue` függvény felelős ezért a transzformációért.

- **Változók (Var):** A függvény egyszerűen kikeresi a nevet az aktuális `SymEnv`-ből.
- **Mező elérés (Field):** A rendszer lekéri a bázis objektum címét, lekérdezi a típusát a `typeof` függvénnyel, megkeresi a tulajdonság eltolódását (`offsetOf`), majd a kettőt összeadja.
- **Tömb indexelés (Index):** A bázis címhez hozzáadódik az index konkrét (kiértékelt) értéke, megszorozva a tömb belső típusának `sizeof` értékével.
- **Dereferencia (Deref):** A rendszer kiértékeli az alapkifejezést. Ha az egy mutató, amely egy címet tartalmaz (`VAddr`), a függvény visszaadja ezt a címet, mint célt.

3.4.8. Implicit Biztonsági Szerződések

A WIC egyik legfontosabb funkciója, hogy a memóriabiztonságot részben automatikusan, a programozó beavatkozása nélkül garantálja. Az explicit szerződések mellett a fordító úgynevezett láthatatlan (implicit) szerződéseket is generál. Ezt az `extractSafety` függvény végzi, amely minden veszélyes műveletet kigyűjt és SMT megkötésekké alakít.

3.4.9. Osztás és tömb túlindexelés elleni védelem

A puffertúlsordulás (buffer overflow) és a nullával való osztás a C programok leggyakoribb sebezhetőségei. Az `extractSafety` rekurzívan bejárja az AST-t, és a következőket teszi:

```

1 extractSafety :: Layout -> t -> Expr -> [(SourcePos, String, Expr)]
2 extractSafety l funcs (Expr pos node) = case node of
3   BinOp "/" e1 e2 ->
4     [(pos, "Division by Zero", genExpr (BinOp "!=" e2 (genExpr
5       (IntLit 0))))]
6     ++ extractSafety l funcs e1 ++ extractSafety l funcs e2
7   Index arr idx ->
8     let baseChecks = extractSafety l funcs arr ++ extractSafety
9       l funcs idx
10    boundsChecks = case typeOf l arr of
11      Just (Array _ (Just (Expr _ (IntLit sz)))) ->
12        [ (pos, "Array Index Out of Bounds", genExpr (
13          BinOp "<" idx (genExpr (IntLit sz))))
14          , (pos, "Array Index Out of Bounds", genExpr (
15            BinOp ">=" idx (genExpr (IntLit 0))))
16        ]
17      _ -> []
18    in boundsChecks ++ baseChecks
19  -- ...

```

3.5. forráskód. Implicit ellenőrzések az `extractSafety` függvényben

Amikor a fordító egy osztást (/) vagy modult (%) talál, automatikusan generál egy kifejezést, amely megköveteli, hogy a nevező (jobb oldali operandus) ne legyen egyenlő 0-val. Tömbök indexelésekor, ha a tömb mérete statikusan ismert, a fordító két megkötést generál: az indexnek nagyobbnak vagy egyenlőnek kell lennie 0-val, és kisebbnek kell lennie a tömb méreténél. Később a szimbolikus motor ezeket a megkötéseket ugyanúgy a Z3 solvernek adja át bizonyításra, mintha a programozó írta volna őket előfeltételként.

3.4.10. Konkrét Kiértékelés (`evalPure`)

Mielőtt az analízátor meghívna a lassú és erőforrás-igényes szimbolikus motort és a Z3-at, megkísérli a kifejezéseket lokálisan, konstans időben egyszerűsíteni és

kiértékelni. Az `evalPure` függvény egy kifejezést kap bemenetként, és a meglévő Heap (memória) alapján megpróbálja konkrét `Value` adattá (pl. `VInt 42`, `VBool True`) alakítani.

Ha az `evalPure` egy olyan változóhoz ér, amelynek értéke függ a futásidejű környezettől (vagy egy argumentumtól, amely még nincs meghatározva), a függvény feladja a próbálkozást, és `VUnknown` értéket ad vissza. Ez jelzi a rendszer számára, hogy a kifejezés szimbolikus (pl. $x + 5$), és át kell adni a Z3 algebrai egyenletmegoldójának.

A mögöttes `applyOp` és `applyUnOp` függvények felelnek az alapvető aritmetikai (összeadás, szorzás) és relációs (kisebb, egyenlő) operátorok C-szabvány szerinti kiértékeléséért az ismert értékek között.

Ezzel a robusztus memóriamoddellel és az implicit szerződések automatikus kivonásával a WIC fordító szilárd alapot biztosít a komplex szimbolikus végrehajtáshoz és az SMT alapú tételbizonyításhoz.

3.5. Szimbolikus Végrehajtás és Tételbizonyítás

A WIC fordító architektúrájának legmeghatározóbb eleme az az innováció, amely a hagyományos fordítóprogramoktól elválasztja: a szimbolikus végrehajtás (*Symbolic Execution*) integrációja. Míg egy megszokott fordító a forráskódot csupán szintaktikailag és típushelyesség szempontjából elemzi, a WIC a kód lehetséges futási útjait matematikai formulákként modellelzi.

3.5.1. A Szimbolikus Motor (`symExec`)

A statikus analízis motorja a `symExec` függvény, amely az Absztrakt Szintaxisfa (AST) utasításait hajtja végre szimbolikusan. Szemben a konkrét kiértékeléssel, ahol egy változó értéke például pontosan 5, a szimbolikus végrehajtás során a változók matematikai szimbólumokat vesznek fel (pl. α), és az ezeken végzett műveletek algebrai kifejezésfáit építik fel.

3.5.2. Az Üzemanyag (`fuel`) Paraméter

Mivel a WIC Turing-teljes nyelv, a szimbolikus végrehajtás során a motor belefuthat a megállási problémába (*Halting Problem*). Egy végtelen ciklus vagy egy

mélyen rekurzív függvényhívás végtelen fává bontaná ki az elemzést, ami a fordító fagyását eredményezné.

Ennek elkerülése érdekében a `symExec` egy úgynevezett `fuel` (üzemanyag) paraméterrel fut. Minden egyes blokkba lépéskor, ciklus iterációkor vagy függvényhíváskor az üzemanyag értéke csökken. Ha a `fuel` eléri a nullát, az elemző megszakítja az adott útvonal feltérképezését, és a hátralévő kifejezéseket biztonsági okokból `VUnknown` (ismeretlen) állapotra redukálja. Ezzel a mechanizmussal a fordító garantálja a statikus analízis terminálását.

3.5.3. Elágazások Szimbolikus Kezelése (If-Else)

Az egyik legkomplexebb feladat a szimbolikus végrehajtás során a feltételes vezérlés-átadások kezelése. Amikor a program egy `If` utasításhoz érkezik, és a feltétel (`cond`) egy szimbolikus (futásidőben eldőlő) érték, az analízátor nem tudja eldönteni, melyik ágon haladjon tovább. Ehelyett a rendszer *kettéágaztatja* a végrehajtást, és mindkét utat bejárja.

3.5.4. Útvonalfeltételek (Path Conditions)

Minden végrehajtási szálnak van egy saját Útvonalfeltétele (`pc` - Path Condition), amely azon logikai állítások konjunkciója, amelyeknek igaznak kell lenniük ahhoz, hogy a program az adott pontra érjen.

- **Igaz ág (*Then*):** A `symExec` létrehoz egy új állapotot, és a `facts` listához hozzáadja a `cond` kifejezést.
- **Hamis ág (*Else*):** A motor egy másik izolált állapotban halad tovább, ahol a `facts` listához a `!(cond)` (negált) kifejezés kerül.

3.5.5. Memóriaállapotok Fúziója (Store Merging)

Miután az `If` és `Else` blokkok elemzése befejeződött, a program folytatódik. Ekkor azonban két eltérő memóriaállapottal (Heap) rendelkezik. A WIC motorjának egyesítenie kell ezeket az állapotokat, hogy a jövőbeli utasítások számára egyetlen koherens képet nyújtson.

Ha egy memóriacímen az érték az igaz ág végén v_{true} , a hamis ág végén pedig v_{false} , a fúzió során a memóriába egy Ternáris operátor (If-Then-Else) kerül.

```

1 mergeStores :: Expr -> Heap -> Heap -> Heap
2 mergeStores cond stT stF =
3     let keys = nub (M.keys stT ++ M.keys stF)
4         getVal st k = fromMaybe VUnknown (M.lookup k st)
5     in M.fromList [ (k, VUnknown) -- A példa kedvéért egyszerűsítve
6                     | k <- keys ]
7 -- A valódi implementáció SMT ITE (If-Then-Else) formulát kepez:
8 -- (Ternary cond (getVal stT k) (getVal stF k))

```

3.6. forráskód. Store fúzió elve a symExec-ben

Például, ha a kód: `if (b) x = 1; else x = 2;`, az egyesítés után `x` értéke a memóriában szimbolikusan: `(b ? 1 : 2)`. Amikor ezt később a Z3 megkapja, pontosan tudni fogja a változó mindkét lehetséges kimenetelét.

3.5.6. Ciklusok (While) Elemzési Stratégiái

A ciklusok jelentik a legnagyobb kihívást a formális verifikáció számára. A WIC fordító két eltérő stratégiát biztosít a ciklusok kezelésére attól függően, hogy a programozó biztosított-e ciklusinvariánst.

1. Stratégia: Bounded Model Checking (BMC)

Ha a `While` ciklushoz nem tartozik invariáns (`Maybe [Expr]` értéke `Nothing`), a WIC a korlátos modellellenőrzést (BMC) alkalmazza. Ilyenkor a `symExec` megkísérli a ciklust kibontja (loop unrolling) egy fix iterációszámig (`unrollLimit`). Egy `while(c) { body }` utasítást az elemző a következőképpen hajt végre: `if(c) { body; if(c) { body; ... } }`.

Ha az `unrollLimit` eléréséig a ciklus befejeződik (például egy fix `for(i=0; i<3; i++)` esetén), az analízis 100%-os pontosságú marad. Ha a ciklus tovább futna, az elemző kénytelen megszakítani a kifejtést, a módosuló változókat ismeretlenre (`VUnknown`) állítani, és egy figyelmeztetést generálni a pontosság elvesztéséről. Ekkor a Transpiler a biztonság érdekében futásidejű asszertációkra kényszerül.

2. Stratégia: Formális Indukció Invariánsokkal

A WIC legfőbb innovációja, hogy a ciklusok mellé írt invariánsokkal képessé válik végtelen vagy futásidőben kiderülő hosszúságú ciklusok matematikai bizonyítására.

Ha rendelkezésre áll egy invariáns (I), a `symExec` a matematikai indukció három lépését szimulálja:

1. **Alapeset (Base Case):** Az analízátor meghívja a Z3-at annak bizonyítására, hogy az I igaz-e rögtön a ciklusba való belépés előtt. Ha nem, a fordítás hibával (`ProvablyFalse`) leáll.
2. **Havoc (Állapot törlése):** Az elemző megkeresi az összes változót a Heap-ben, ami a ciklusmagban módosulhat, és az értéküket `VUnknown`-ra cseréli. Erre azért van szükség, hogy a bizonyítás ne függjön az első iteráció konkrét értékeitől, hanem általánosan minden iterációra igaz legyen.
3. **Induktív Lépés (Inductive Step):** A motor feltételezi (hozzáadja a `facts` listához), hogy a havoc-olt változókkal I és a ciklus feltétele (`cond`) igaz. Ezután szimbolikusan lefuttatja a ciklusmagot egyetlen egyszer. A mag végén ismét lekéri a Z3-tól, hogy igaz maradt-e I . Ha a válasz `ProvenTrue`, a ciklus bizonyítottan biztonságos, és nincs szükség loop unrollingra.

3.5.7. SMT Solver Integráció (proveZ3)

Amikor az analízátor felépített egy biztonsági megkötést (pl. az implicit szerződésekéből kinyert nevező $\neq 0$ formulát), a matematikai kifejezést a Microsoft Z3 Theorem Prover belső formátumává kell alakítania.

3.5.8. A `translateZ3` függvény

A WIC típusrendszere szigorúan illeszkedik a Z3 beépített elméleteihez (*Sorts*).

- A `Type Int` a Z3 `mkIntSort`-ra fordul.
- A `Type Bool` a Z3 `mkBoolSort`-ra fordul.
- A WIC memóriacímek feloldására az elemző Z3 `Array` formulákat alkalmaz, ahol az `index` egy cím, az érték pedig a tárolt adat.
- Az idegen függvényhívásokat az úgynevezett "Uninterpreted Functions" koncepciójával fedi le az `mkFuncDecl` segítségével. A Z3 nem ismeri a függvény belső logikáját.

3.5.9. A Döntési Algoritmus (proveZ3)

A SMT solver egy formuláról azt tudja eldönteni, hogy kielégíthető-e (*Satisfiable*). A WIC a "cáfolat általi bizonyítás" (Proof by Contradiction) elvét használja. Legyen a bizonyítandó tulajdonság P , és az aktuális útvonalfeltételek konjunkciója PC .

A `proveZ3` algoritmus háromféle állapotot tud visszaadni:

1. **ProvenTrue (Biztosan Igaz):** A fordító beadja a Z3-nak a $PC \wedge \neg P$ formulát. Ha a Z3 válasza UNSAT (kielégíthetetlen), az azt jelenti, hogy soha, semmilyen bemenetre nem tud P hamis lenni. A tétel bizonyítást nyert, a `Transpiler` kihagyja a futásidejű ellenőrzést (Zero-cost abstraction).
2. **ProvablyFalse (Biztosan Hamis / Fatal Error):** Ha a fenti vizsgálat SAT (tehát lehet hamis), a fordító megkísérli a $PC \wedge P$ formulát (a pozitív esetet) kielégíteni. Ha ez UNSAT, az azt jelenti, hogy az adott kódrészlet *sosem* működhet helyesen (pl. garantált nullával osztás van benne). Ekkor a WIC fordítási hibával azonnal leáll.
3. **RuntimeCheck (Eldönthetetlen / Unknown):** Ha mind a pozitív, mind a negatív eset SAT, akkor a megkötés futásidejű adatoktól függ (pl. egy konzolról beolvasott érték). Ez a kategória foglalja magába azt is, ha a Z3 `Undef` (Timeout vagy nem lineáris aritmetika) választ ad. Ilyenkor a fordító az "Unknowns" listához adja a megkötést, amit a generátor később egy `assert(P)` hívásként ágyaz a C kódba.

Ez a háromállapotú logika biztosítja, hogy a WIC soha ne fogadjon el csendben hibás kódot, de ne is gátolja a fejlesztést olyan esetekben, amikor az analízis korlátaiba ütközik. A rendszer ezáltal a formális verifikáció precizitását a C nyelv rugalmasságával ötvözi.

3.6. Kódgenerálás és a WIC Transzpiler

A fordítási pipeline utolsó fázisa a kódgenerálás, amelyért a WIC architektúrájában a `Transpiler.hs` modul felel. Miután a `Parser` felépítette az Absztrakt Szintaxisfát (AST), az `Analyzer` pedig a Z3 SMT solver segítségével végrehajtotta a szimbolikus verifikációt, a rendszernek egy futtatható állományt kell előállítania.

A WIC tudatos tervezési döntése volt, hogy nem közvetlenül gépi kódot (Assembly) vagy LLVM IR-t generál, hanem egy platformfüggetlen, szabványos C99 forráskódot. Ez a transzpilációs (source-to-source compiler) megközelítés lehetővé teszi a meglévő, magasan optimalizált C fordítók (mint a GCC vagy Clang) erejének kiaknázását, miközben a WIC réteg garantálja a formális memóriabiztonságot.

3.6.1. A Kimeneti Fájl Felépítése és C-interoperabilitás

A `transpile` függvény a generátor belépési pontja, amely összeállítja a végső C fájlt. A WIC nyelv tervezésekor elsődleges szempont volt a meglévő C ökoszisztémával való kompatibilitás (interoperabilitás). Ennek érdekében a fordító képes befogadni és változatlan formában továbbadni a C preprocesszor direktívákat.

```
1 transpile :: [String] -> [Expr] -> [TopLevel] -> String
2 transpile directives unknowns ast =
3     let customDirectives = unlines directives
4         headers = "#include <stdio.h>\n#include <stdlib.h>\n"
5                 ++ "#include <stdbool.h>\n#include <assert.h>\n\n"
6                 ++ "// --- User Directives ---\n"
7                 ++ customDirectives
8                 ++ "// -----\n"
9     in headers ++ "\n\n" ++ intercalate "\n\n" (map (
        transpileTopLevel unknowns) ast)
```

3.7. forráskód. A kimeneti fájl fejléceinek összeállítása

A transzpiler automatikusan beilleszti az alapvető standard könyvtárakat. Ezek közül a legfontosabb az `<assert.h>`, amely a futásidejű biztonsági mechanizmusok alapját képezi. A `Main.hs` által az elemzés előtt leválasztott direktívák (`directives`) ezt követően kerülnek a fájlba, biztosítva, hogy a WIC kódból hívni lehessen külső C könyvtárak (pl. `math.h` vagy operációs rendszer szintű API-k) függvényeit.

3.6.2. Típusok és Globális Deklarációk Leképezése

A legfelső szintű AST elemek (`TopLevel`) konverziója a `transpileTopLevel` függvényben történik. A WIC típusrendszere szorosan követi a C memóriamodelljét, így a leképezés viszonylag direkt, de a szintaktikai különbségeket a generátornak kell áthidalnia.

3.6.3. Összetett Adatszerkezetek (Struct, Union, Enum)

A rekord típusok leképezése egy az egyben megtartja a C szemantikát.

```

1  DefStruct name fields ->
2      "struct " ++ name ++ " {\n" ++
3      unlines (map (\(t, n) -> "    " ++ transpileDecl t n ++ ";"
4      ) fields) ++
      "};"

```

3.8. forráskód. Struktúrák transzpilálása C kódra

Látható, hogy a WIC-ben definiált struktúrákból szabványos C `struct` lesz. Ugyanez a logika érvényesül a `Union` és az `Enum` típusoknál is, biztosítva a memóriakiosztás és az offset-számítások determinizmusát, amit az analízátor is feltételezett.

3.6.4. Minősítők (Qualifiers) és Deklarációk

A WIC támogatja a statikus minősítőt (`Static`), amely a generált C kódban a `static` kulcsszóval jelenik meg, korlátozva az adott globális változó vagy függvény láthatóságát (linkage) az adott fordítási egységre. A változódeklarációk generálása-kor (`transpileDecl`) a rendszer figyelembe veszi a mutatókat (`Ptr`) és a konstansokat (`ConstT`), helyesen elhelyezve a `*` és `const` kulcsszavakat.

3.6.5. Kifejezések és Utasítások Fordítása

A függvények törzsének konverziója a `transpileStmt` és `transpileExpr` függvényeken keresztül valósul meg.

3.6.6. Kifejezések (Expressions)

A `transpileExpr` rekurzívan bejárja a kifejezésfát. A legtöbb operátor közvetlenül megfeleltethető a C megfelelőjének:

- **Memória operátorok:** A `Deref e` átalakul `(*e)`-vé, az `AddrOf e` pedig `(&e)`-vé. A zárójelezés bőséges használata a generált kódban garantálja, hogy a WIC elemzőjének precedenciaszabályai pontosan megőrződjenek a C fordító számára is.
- **Mezőelérés:** A `Field e p` formátum a megszokott `e.p` formára fordul.

- **Belső AST csomópontok:** Az `ArrayUpdate` egy olyan speciális WIC AST csomópont, amelyet kizárólag a Z3 SMT Array elméletének felépítéséhez használ a memóriamodell. Mivel ennek nincs C nyelvű megfelelője (a C mutatóaritmetikával módosít tömböt), a transzpiler egy hibakommentet generál, ha ilyennel találkozik.

3.6.7. Dinamikus Biztonsági Háló: Asszertációk Injektálása

A WIC "Zero-cost abstraction" modelljének és transzpilációs stratégiájának legkritikusabb lépése a dinamikus futásidejű védőháló kialakítása. Amint azt az *Analyzer* fejezetben láttuk, ha a Z3 solver mindent képes matematikailag bizonyítani (*ProvenTrue*), a kód nem tartalmaz overheadet. Azonban vannak olyan esetek, amikor az SMT bizonyítás a fizikai határokba ütközik (pl. nemlineáris aritmetika, futásidőben konzolról beolvasott adatok).

Ezeket a bizonyíthatatlan megkötéseket az analízátor az `unknowns` (ismeretlenek) listájában adja át a transzpilernek.

3.6.8. Szerződéses futásidejű ellenőrzése

A `transpileFunc` és a `transpileStmt` függvények felelnek az `unknowns` kifejezések C `assert()` utasításokká alakításáért és megfelelő helyre való beillesztéséért.

1. **Előfeltételek (Preconditions):** Ha a hívó fél által biztosítandó `funcInvars` feltételek között akad bizonyíthatatlan, a transzpiler ezeket *közvetlenül a függvény törzsének legelejére* szúrja be.

```
1   Int divide(Int a, Int b) {  
2       assert(b != 0); // Injektált függőség az unknowns listá  
        ból  
3       return a / b;  
4   }
```

3.9. forráskód. Generált előfeltétel assertáció C-ben

2. **Utófeltételek (Postconditions):** Az utófeltételek garantálása a legnehezebb feladat kódgenerálás szempontjából, mivel a függvény több ponton is visszatérhet (`Return` utasítások). Amikor a transzpiler egy `Return` utasításhoz ér a `transpileStmt` logikában, először kiszámítja a visszatérési értéket (`res`),

ezt egy temporális változóba menti, lefuttatja rajta a visszakapott utófeltétel megkötéseket (`postChecks`), majd meghívja az igazi `return`-t.

3. **Ciklus Invariánsok és Implicit Védelmek:** Ha az analízátor a *Bounded Model Checking* (BMC) során kifogy az "üzemanyagból", vagy egy implicit tömb túlindexelés ellenőrzése eldönthetetlen marad, az asszertáció pontosan a probléma (osztás, indexelés, ciklusmag kezdete) elé injektálódik a `Block` generálási szakaszában.

Ezzel az architektúrával a WIC sikeresen ötvözi a statikus verifikáció teljesítménybeli előnyeit a futásidejű ellenőrzések biztonságával.

3.7. Szoftvertesztelés és Validáció

Egy fordítóprogram, különösen egy SMT alapú formális verifikációs eszköz fejlesztése rendkívül érzékeny feladat, ahol a legapróbb hiba is hamis biztonságérzetet (false positive), vagy indokolatlan fordítási hibákat (false negative) eredményezhet. Ennek elkerülése érdekében a WIC projekt a Haskell ökoszisztéma egyik legfejlettebb BDD (Behavior-Driven Development) tesztelési keretrendszerét, a `Hspec`-et alkalmazza.

3.7.1. A Tesztelési Keretrendszer Architektúrája

A projekt konfigurációs fájlja (`WIC.cabal`) egy dedikált `test-suite` blokkot tartalmaz, amely biztosítja, hogy a tesztek futtatásakor mind a `Parser`, mind az `Analyzer`, mind a `Transpiler` modulok betöltődjenek. A fő executable és a tesztkörnyezet a modern `GHC2024`-re épül, és a tesztkörnyezet függőségei között szerepel a `hspec`.

A tesztelés fókuszpontja a `Test.hs` fájl, amely nem csupán az AST építésének helyességét, de a `Z3` solver válaszait (Unknowns, Violations) is integráltan teszteli.

3.7.2. Az AST Pozícióttörlési Mechanizmusa

A szintaktikai elemző egységtesztelése során egy komoly akadályba ütközünk. A `Megaparsec` által előállított AST (`Expr` és `Stmt` csomópontok) minden eleme egy

`SourcePos` adattal van kiegészítve. Ez a pozíció tartalmazza a fájlnevet, a sort és az oszlopot.

Amikor egy tesztetben ellenőrizni akarjuk, hogy a `2 + 3` kifejezés helyesen lett-e lefordítva `BinOp "+" (IntLit 2) (IntLit 3)`-má, a Haskell standard egyenlőségvizsgálata `(==)` `False`-t adna vissza, mivel a kézzel felépített "várt" (expected) fának más pozíciói lesznek, mint amit a fájl beolvasása során a parser generált.

Ennek áthidalására a fejlesztői környezet implementálja a *Position Stripping* (Pozícióttörlés) logikát.

```

1 clearPos :: SourcePos
2 clearPos = initialPos "test-input"
3
4 cP_E :: Expr -> Expr
5 cP_E (Expr _ node) = Expr clearPos $ case node of
6     Index e1 e2 -> Index (cP_E e1) (cP_E e2)
7     Call n args -> Call n (map cP_E args)
8     Field e p -> Field (cP_E e) p
9     BinOp op e1 e2 -> BinOp op (cP_E e1) (cP_E e2)
10    _ -> node

```

3.10. forráskód. Position Stripping a Test.hs modulban

A `cP_E` (Clear Position Expression) és a hozzá tartozó `cP_T` (Clear Position Type) függvények rekurzívan bejárják a vizsgálandó AST-t, és minden `SourcePos`-t felülírnak egy közös `clearPos` értékre ("test-input", 1. sor, 1. oszlop). Ezzel a transzformációval az AST strukturális tesztelhetősége (strukturális egyenlőség) akadálytalanul megvalósul.

3.7.3. Szimbolikus Analízis és SMT Logika Validációja

A tesztelési modul legfontosabb része a formális verifikáció viselkedésének ellenőrzése. A WIC elemzője háromféle kimenetet adhat egy feltételre: *ProvenTrue*, *ProvablyFalse*, vagy *Unknown*. A `Test.hs` mindhárom esetet szigorú forgatókönyvekkel fedi le.

3.7.4. Negatív tesztek (Provably False)

Egy megbízható fordítónak meg kell állnia, ha bizonyíthatóan elkerülhetetlen hibát (*Fatal Error*) talál. A tesztek ellenőrzik, hogy az elemző észleli-e a triviális

ellentmondásokat.

```
1 Void requirePositive(Int val)(val > 0) { return; }
2 Int main() {
3     Int x = -5;
4     requirePositive(x);
5     return 0;
6 }
```

3.11. forráskód. Szerződésszegés tesztelése

Ebben a tesztetben az analízátor futtatása során az **unknowns** lista üres kell, hogy maradjon (mert a hiba nem futásidejű adattól függ), a hibák (**errors**) listájában pedig szerepelnie kell a "Precondition Failed" üzenetnek. A teszt ezt a struktúrát a **shouldBe** operátorral validálja.

3.7.5. Nemlineáris Aritmetika és Eldönthetetlenség

A WIC architektúrájának hibrid jellege (ami miatt a transzpiler **assert**-et tud injektálni) a legnehezebben bizonyítható SMT feladatoknál, a nemlineáris aritmetikánál (pl. szorzás ismeretlenekkel) mutatkozik meg. A tesztelési keretrendszer ezt a Nagy Fermat Tételével (*Fermat's Last Theorem*) validálja.

```
1 Void requiresMagic(Int a, Int b, Int c)
2     ((a*a*a + b*b*b) != (c*c*c))
3 { return; }
```

3.12. forráskód. Eldönthetetlen probléma tesztelése Fermat Tételével

Mivel a köbös (nemlineáris) egyenlőségek a SMT solverek számára gyakran eldönthetetlenek (a Z3 is feladja bizonyos idő után, vagy nem tudja bizonyítani a végtelen halmazon), a teszt azt várja el, hogy a fordítás **ne álljon le hibával** (az **errors** lista üres legyen). Ehelyett a vizsgált feltétel be kell, hogy kerüljön az **unknowns** listába: $((a * a) * a) + ((b * b) * b) \neq ((c * c) * c)$. Ez bizonyítja, hogy az **Analyzer** helyesen halasztotta futásidőre a komplex matematikai számítást.

Ezek a robusztus tesztelési stratégiák garantálják, hogy a WIC C kódgenerátora és SMT alapú memóriamodellje stabil, konzisztens, és ami a legfontosabb, a megalkotott elméleti háttérnek megfelelően viselkedik az ipari környezetben történő alkalmazás során is.

4. fejezet

Összegzés

Ezen szakdolgozat keretében egy olyan innovatív fordítóprogram-architektúra lett bemutatva, amely az imperatív programozás rugalmasságát ötvözi a formális verifikáció biztonságával. A kifejlesztett WIC nyelv és a hozzátartozó transzpiler bizonyította, hogy a statikus analízis és a szimbolikus végrehajtás integrálása a fordítási folyamatba nem csupán elméleti lehetőség, hanem a gyakorlatban is megvalósítható, hatékony védelmi vonal a szoftverhibák ellen.

4.1. Eredmények összefoglalása

A projekt megvalósítása során a következő mérföldköveket sikerült elérni:

- **Robusztus szintaktikai elemző:** A Megaparsec könyvtár segítségével egy olyan kézzel írt parser készült, amely képes kezelni az összetett C-stílusú típusrendszert és a nyelv egyedi, szerződésalapú (szerződések, invariánsok) kiterjesztéseit.
- **Szimbolikus végrehajtó motor:** Implementálva lett egy Haskell alapú analízátor, amely képes a program állapotának absztrakt modellezésére, az útvonalfeltételek (*Path Conditions*) gyűjtésére és a memóriaállapotok fúziójára az elágazások mentén.
- **SMT integráció:** Integrálásra került a Microsoft Z3 Theorem Prover a fordítóba. A rendszer képes a WIC kifejezéseket automatikusan SMT formulákká alakítani és háromállapotú logika segítségével dönteni azok helyességéről.

- **Hibrid kódgenerálás:** A transzpiler megvalósítja a "Zero-cost abstraction" elvét: a bizonyítottan biztonságos kódrészletekből eltávolítja az ellenőrzéseket, míg a bizonyíthatatlan kényszereket dinamikus `assert` utasításokká alakítja, fenntartva a biztonság és a teljesítmény egyensúlyát.

A fejlesztés során alkalmazott tesztelési stratégia, különös tekintettel a *Position Stripping* mechanizmusra és a komplex matematikai (pl. nemlineáris aritmetikai) esetek vizsgálatára, igazolta, hogy a rendszer stabilan és előrejelezhetően kezeli a szélsőséges programozói forgatókönyveket is.

4.2. Jövőbeli fejlesztési lehetőségek

Bár a koncepcióigazolás sikeres volt, a WIC nyelv és fordító továbbfejlesztése számos izgalmas irányt kínál:

- **Komplex mutatóaritmetika:** A jelenlegi memóriamodell kiterjesztése a mutatók közötti bonyolultabb műveletek (pl. pointer casting, pointer-to-int konverzió) precízebb modellezésére.
- **Lebegőpontos algebra:** Az IEEE 754 szabvány szerinti lebegőpontos számítások SMT alapú verifikációjának bevezetése, amely különösen fontos lenne tudományos számítások során.
- **Optimalizált cikluskezelés:** A cikluskifejtés (*loop unrolling*) intelligens, heurisztika alapú vezérlése a *Bounded Model Checking* hatékonyságának növelése érdekében.
- **Standard könyvtár támogatása:** Egy biztonságos standard könyvtár (libc-szerű) kialakítása, ahol minden függvény rendelkezik formálisan bizonyított elő- és utófeltételekkel.
- **A C/C++ szabvány támogatásának bővítése:** A WIC nyelv szintaktikai és szemantikai elemeinek kiterjesztése a C++ egyes jellemzőire (pl. osztályok, kivételek) a szélesebb körű alkalmazhatóság érdekében.

Zárogondolatként kijelenthető, hogy a WIC fordító architektúrája rávilágított arra, hogy a modern fordítóprogramoknak túl kell lépniük a puszta kódgeneráláson. A formális módszerek automatizált alkalmazása a jövőben alapvető elvárássá válhat

a kritikus szoftverrendszerek fejlesztése során, ahol a hibák elkerülése nem csupán gazdasági, hanem gyakran biztonsági kérdés is.

Köszönetnyilvánítás

Szeretném megköszönni Bense Viktor témavezetőmnek a folyamatos támogatást, értékes tanácsokat és a szakmai irányítást, amely nélkül ez a projekt nem valósulhatott volna meg. Köszönöm továbbá a Z3 SMT solver fejlesztőinek, hogy egy ilyen erőteljes eszközt tettek elérhetővé a közösség számára, amely kulcsfontosságú volt a WIC fordító architektúrájának megvalósításában.

Irodalomjegyzék

- [1] Leonardo de Moura és Nikolaj Bjørner. „Z3: An Efficient SMT Solver”. *Tools and Algorithms for the Construction and Analysis of Systems*. Szerk. C. R. Ramakrishnan és Jakob Rehof. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, 337–340. old. ISBN: 978-3-540-78800-3.
- [2] *International Standard*. [https : / / www . gwduan . com / web / computer / languages/c/c99 . pdf](https://www.gwduan.com/web/computer/languages/c/c99.pdf). [Online; Accessed 2026.04.30.] 1999.

Forráskódjegyzék

2.1. Csomagok telepítése Ubuntu alatt	7
2.2. <code>cabal.project.local</code> beállítása Windows alatt	8
2.3. Változók és konstansok deklarálása	12
2.4. Tömbök és mutatók használata	12
2.5. Függvény egy egyszerű előfeltétellel	15
2.6. Függvény utófeltétellel	15
2.7. A ciklus invariáns gyakorlati alkalmazása	16
2.8. Automatikus index ellenőrzés	17
3.1. Az <code>AnalyzerState</code> felépítése	23
3.2. Operátor táblázat a <code>Parser.hs</code> -ben	25
3.3. Az <code>AnalyzerState</code> felépítése	28
3.4. Az <code>offsetOf</code> függvény	29
3.5. Implicit ellenőrzések az <code>extractSafety</code> függvényben	31
3.6. Store fúzió elve a <code>symExec</code> -ben	34
3.7. A kimeneti fájl fejléceinek összeállítása	37
3.8. Struktúrák transzpilálása C kódra	38
3.9. Generált előfeltétel assertáció C-ben	39
3.10. Position Stripping a <code>Test.hs</code> modulban	41
3.11. Szerződészegés tesztelése	42
3.12. Eldönthetetlen probléma tesztelése Fermat Tételével	42